

Here's a plain-English guide to your parser, written for teammates. I won't change any code—this just explains what's there and how to use it.

What this program does (at a glance)

- **Tokenises** a MiniLisp-style input string (e.g. (+ 2 3)).
 - **Parses** it with a **table-driven LL(1) parser** (predictive, 1-token lookahead).
 - **Builds** a simple tree (nested Python lists/values) like ["PLUS",2,3].
 - **Runs tests** and writes a human-readable results file (Input:/Output: blocks).
-

Unicode “gotchas” (very important)

- Minus is `–` (U+2212), **not** the normal hyphen `-`.
 - Multiply is `×` (U+00D7).
 - Lambda is `λ` (U+03BB).
 - Let/definition is `≐` (U+225C).
If you type ASCII `-` or `*`, the lexer will complain. Use UTF-8.
-

Modules, constants, and data

`uniCodeTable` (dictionary)

What it is: A map from token names to the exact Unicode code points they match (or lists of code points, e.g. all digits).

Why we need it: The lexer consults this to recognise characters like `(`, `×`, `λ`, digits, letters, etc.

The Lexer (turns text into tokens)

`class LexicalAnalyser`

Holds all the functions used to scan the raw input into a token list.

lookupChar(ch, table)

- **Input:** a single character `ch`, and the `uniCodeTable`.
- **Output:** a string key like "PLUS", "letters_lower", "NUM", or None if unknown.
- **Use:** Primitive helper—everything else calls this to classify characters.

TokenType(ch)

- **Input:** one character.
- **Output:** the **token type name** for single-character tokens ("PLUS", "LPAREN", ...), or "NUMBER" if it's a digit, or None if unrecognised.
- **Use:** Decides what kind of token we're looking at when scanning.

isNum(ch)

- **True** if `ch` is a digit 0–9.

isLetter(ch)

- **True** if `ch` is an English letter **or underscore** (note: this is slightly more permissive than the spec—underscore counts as a letter here).

isWhitespace(ch)

- **True** for space, tab, newline.

scanNumber(source, index)

- **Input:** the whole string and the current index (pointing at a digit).
- **Output:** (`next_index`, `value`) where `value` is an int or sometimes a float.
- **Behaviour:**
 - o If it starts with 0:
 - If followed by `.` and at least one digit, it reads a **fraction like 0.123** and returns a **float**.
 - Otherwise returns the number 0 and stops (so "00" becomes 0 then another 0 on the next scan).

- o If it starts with 1–9, it reads all following digits as an **int**; if a . appears after that it **errors** (“Invalid number format”).
- **Gotcha:** This allows 0.xxx floats, but not 12.3. Also "00" is handled as two separate zeros.

scanIdentifier(source, index)

- **Input:** string and index (pointing at a letter or underscore).
- **Output:** (next_index, name) where name is the identifier substring.
- **Rules:** First char must be “letter” (here: A–Z, a–z, or underscore). After that, letters or digits are allowed.

analyse(input: str)

- **Input:** the entire program string.
 - **Output:** a list of tokens like [("PLUS", "+"), ("NUMBER", 2), ... , ("EOF", None)].
 - **What it does:**
 - a. Skips whitespace.
 - b. If digit → calls scanNumber and emits ("NUMBER", value).
 - c. If letter/underscore → calls scanIdentifier and emits ("IDENTIFIER", name).
 - d. If one-char token (e.g. (,), +, ×, ?, λ, ≐) → emits that token type.
 - e. Otherwise → raises an error with the character and index.
 - f. Appends final ("EOF", None).
-

Grammar & Parse Table

Non-terminal names

- NT_PROGRAM, NT_EXPRESSION, NT_PAREN (and NT_ARGTAIL is defined but **not used** here).

Builder helpers (build the output tree)

- skip(child) – returns the first element; used when we don't need a wrapper node.
- num(child) – returns the number value.
- ident(child) – returns the identifier string.
- op2(name) – factory that returns a function combining two child expressions into [name, left, right].
- conditional(child) – returns ["COND", cond, then, else].
- lamda(child) – returns ["LAMBDA", ident, body].
- let(child) – returns ["LET", ident, expr1, expr2].
(These are attached to productions to construct the parse tree once children are reduced.)

The parse table PT and add(...)

- PT is a dict keyed by (non_terminal, lookahead_token_type).
- add(nt, lookahead_list, rhs, builder) fills PT so the parser knows which **RHS** (right-hand side) to expand and which **builder** to run after.

Key productions encoded:

- PROGRAM $\rightarrow$ EXPRESSION
- EXPRESSION $\rightarrow$ NUMBER | IDENTIFIER | '(' PAREN ')'
- PAREN $\rightarrow$ PLUS EXPR EXPR
- PAREN $\rightarrow$ MINUS EXPR EXPR
- PAREN $\rightarrow$ MULT EXPR EXPR
- PAREN $\rightarrow$ EQUALS EXPR EXPR
- PAREN $\rightarrow$ CONDITIONAL EXPR EXPR EXPR
- PAREN $\rightarrow$ LAMBDA IDENTIFIER EXPR
- PAREN $\rightarrow$ LET IDENTIFIER EXPR EXPR

Not included: application/grouping (expr expr*). So (x) or (x 1 2) are **errors** by design.

The Parser (turns tokens into a tree)

`class parseError(Exception)`

- Custom exception type for parse failures.

`class LL1Parser`

`parse(tokens)`

- **Input:** token list from the lexer.
- **Output:** one value: the full parse tree (numbers/strings or nested lists).
- **How it works (predictive LL(1)):**
 - o Starts with a stack containing PROGRAM.
 - o Repeatedly:
 - If the **top** of the stack is a **non-terminal**, look up (non_terminal, current_token_type) in PT:
 - Push a special ("@reduce", builder, mark) to know where children start on the value stack.
 - Push the RHS symbols **in reverse** so they're processed in order.
 - If the top is a **terminal** (e.g. "NUMBER", "PLUS") it must match the current input token's type:
 - If it's NUMBER or IDENTIFIER, push the token's **value** (like 42 or "x") onto the value stack.
 - Advance the input.
 - If the top is a ("@reduce", builder, mark):
 - Pop all children collected since mark, call builder(children) to make a node, and push that node back.
 - If there's no matching table entry, raise a parseError describing the unexpected token and what was expected.

- o At the end it verifies: last token is EOF, all input consumed, and exactly one value remains (the parse tree). Returns it.
-

Convenience & I/O

`parseInput(source: str)`

- **What it does:** Tokenises (analyse) then parses (`LL1Parser.parse`).
- **Return:** the parse tree (Python types/lists), **not** JSON text.

`compactJSON(obj)`

- **Helper:** Serialises any Python object to **compact JSON** (no spaces), for neat output like `["PLUS",2,3]`.

TESTS (list of sample cases)

- A tuple list of (name, input, shouldFail) including basics (42, x), operators, nested cases, λ , $\underline{a}$, and deliberate error cases (missing paren, extra arg, etc.).
- Note: it still *lists* grouping/application test names, but your current logic treats (x) / (x 1 2) as **errors**, which matches your grammar.

`main()`

- **Creates** `./parser_tests_output/`.
- **Runs** tests once to count pass/fail (there's a small bug in the first loop's except that references output before it's set—just be aware).
- **Writes** a human-readable file `results_output.json` (despite the name, it writes **text**, not a JSON array) with repeated blocks:

vbnetCopy codeInput: <the input>

Output: <either a compact JSON tree or an ERROR: ... message>

- **Prints** a summary line to stdout: Tests passed: X, Tests failed: Y.

`if __name__ == "__main__": demo prints`

- Tries several sample inputs and prints either the raw Python tree or an error message.
-

Typical examples (what to expect)

- $(+ 2 3) \rightarrow ["PLUS", 2, 3]$
 - $(\times x 5) \rightarrow ["MULT", "x", 5]$
 - $(+ (\times 2 3) 4) \rightarrow ["PLUS", ["MULT", 2, 3], 4]$
 - $(? (= x 0) 1 0) \rightarrow ["COND", ["EQUALS", "x", 0], 1, 0]$
 - $(\lambda x x) \rightarrow ["LAMBDA", "x", "x"]$
 - $(\triangleq y 10 y) \rightarrow ["LET", "y", 10, "y"]$
 - $(x) \rightarrow \text{ERROR}$ (application/grouping not supported)
 - $(x 1 2) \rightarrow \text{ERROR}$ (application not supported)
 - $(+ 2 \rightarrow \text{ERROR}$ (missing right parenthesis)
 - $(+ 2 3 4) \rightarrow \text{ERROR}$ (too many arguments)
-

When to use which function (cheat sheet)

- **Parse a single string to a tree:** `parseInput("...")`
 - **Just lex (debugging):** `LexicalAnalyser.analyse("...")`
 - **Compact printing of a Python tree as JSON:** `compactJSON(tree)`
 - **Run the full test suite and write outputs:** run the script directly (python yourfile.py)
-

Known behavioural quirks (as implemented)

- **Identifiers** accept underscore (e.g. `_x`, `x_y`)—broader than the stricter spec.
- **Numbers** allow `0.xxx` floats but not `12.3`; `"00"` is tokenised as `0` then `0`.
- **Application** is intentionally **not** recognised; only the listed operator/construct forms inside parentheses are valid.

If you want this turned into a one-page internal doc or added to a README, say the word and I'll format it that way.